

Coverage-Preserving Compilation of Normative and Empirical Security Knowledge

Pedro Farinha

Independent Researcher

pedro.farinha@shiftleft.pt

github.com/pedrofarinhaatshiftleftpt

Supported by Shiftleft - Secure Software Engineering, lda.

Abstract

Application security knowledge exists in two forms that are rarely integrated: normative knowledge (what frameworks prescribe) and empirical knowledge (what practitioners have learned). Frameworks such as SSDF, ASVS, and SLSA define requirements but cannot capture implementation specifics, organizational context, or practices that predate formalization. Practitioner documentation captures these but lacks cross-framework traceability and coverage awareness.

We present a method for compiling both knowledge types into a unified, coverage-aware model. The method uses a normalized ontology (AppSec Core [1]) as the integration substrate: normative requirements from external frameworks and empirical content from a practitioner corpus are independently mapped to a shared set of control objectives, then compared to assess coverage, identify gaps, and detect empirical extensions (content that exists in practice but is not required by any framework).

We apply the method to a case study: a 15-chapter practitioner-authored security manual (4,139 structural units) compiled against 5 external frameworks (91 mapped items). The compilation reveals: (a) 78% direct observed coverage, rising to 95% of in-scope items after claim-gap resolution — with only 4 genuine content gaps; (b) a claim gap phenomenon — most apparent gaps are traceability failures rather than content deficiencies; and (c) measurable empirical delta — the practitioner corpus contains content that extends beyond framework requirements, particularly in operational mechanisms and evidence patterns.

We contribute: (1) a dual-source knowledge compilation method with explicit coverage semantics; (2) the formal distinction between *claim gaps*, *content gaps*, and *cross-reference gaps*; (3) empirical evidence that practitioner knowledge substantially covers and extends normative frameworks; and (4) the *ontology-driven refinement loop*, where compilation outputs improve the source corpus.

Keywords: knowledge compilation, application security, normative knowledge, empirical knowledge, coverage analysis, security requirements, ontology

1. Introduction

Organizations building application security programs draw on two distinct knowledge sources. Normative knowledge — what external frameworks prescribe — provides structured requirements: NIST SSDF [2] defines 20 development practices, OWASP ASVS [3] specifies 443 verification requirements, SLSA [4] mandates build integrity properties. Empirical knowledge — what practitioners have learned — provides operational depth: coding guidelines, CI/CD security configurations, container hardening procedures, threat modeling patterns, and training curricula accumulated through real engagements.

These two knowledge types are complementary but rarely integrated. Frameworks provide breadth and regulatory alignment; practitioner documentation provides depth and operational specificity. Neither alone is sufficient:

- Frameworks without empirical grounding produce compliance checklists disconnected from implementation reality
- Practitioner knowledge without framework alignment produces operationally rich documentation with no coverage assurance

The challenge is to compile both into a unified model that preserves the coverage properties of frameworks while retaining the operational richness of practitioner knowledge — and to do so with measurable, auditable properties.

The Compilation Problem

Knowledge compilation in this context is not simple merging. It requires:

1. Normalization: projecting heterogeneous frameworks and practitioner content into a shared canonical representation where semantic equivalence can be evaluated
2. Deduplication: recognizing when a framework requirement and a practitioner practice address the same concern
3. Coverage assessment: determining which normative requirements are satisfied by the empirical corpus
4. Gap identification: distinguishing genuine content gaps (nothing exists) from visibility gaps (content exists but is not exposed)
5. Delta detection: identifying empirical content that extends beyond what any framework requires

This paper presents a method that addresses all five, using a normalized ontology (AppSec Core [1]) as the integration substrate.

Contributions

1. A dual-source compilation method that integrates normative and empirical security knowledge through a shared ontological layer, with explicit coverage semantics.
2. The formal distinction between claim gaps, content gaps, and cross-reference gaps — three categories of apparent non-coverage with different remediation strategies.
3. Empirical evidence from a case study (15-chapter manual, 4,139 units, 5 frameworks, 91 mapped items) showing that practitioner knowledge substantially covers and extends normative frameworks.
4. The ontology-driven refinement loop: compilation outputs (gap classifications, source references) feed back into the empirical corpus, improving its traceability surface.

2. Related Work

2.1 Security Framework Alignment

Organizations typically align practices with frameworks through manual mapping: reading each framework's requirements and determining whether internal documentation addresses them. Several tools and methodologies support this (e.g., NIST SP 800-53 overlays [5], compliance matrices). These operate on a per-framework basis with no shared semantic model across frameworks — the quadratic mapping

problem identified in [1].

2.2 Knowledge Compilation in SE

Knowledge compilation — transforming dispersed knowledge into a unified, queryable representation — has been studied in software engineering through experience factories [6] and knowledge management frameworks. Basili et al. [6] established the paradigm of accumulating and packaging practitioner experience. Our work extends this by adding *coverage-aware* compilation: the unified representation not only stores knowledge but enables coverage assessment against external reference sets.

2.3 Requirements Traceability

Ramesh and Jarke [8] established reference models for requirements traceability, identifying pre-requirements, requirements, and design as distinct traceability layers. Goknil et al. [7] further formalized trace relation semantics for requirements models. Our work introduces an additional dimension to this literature: epistemic weight — different document types carry different authority, and coverage analysis must account for this stratification. A normative requirement (strong weight) and an operational add-on (medium weight) both contribute to coverage, but differently.

2.4 Gap Analysis in Security

Gap analysis in security compliance typically compares an organization’s documented controls against a framework’s requirements. Without a structured corpus index, a common working assumption is that absence of documentation implies absence of practice. Empirical studies of traceability practice show that this assumption frequently fails: practitioners omit or defer explicit traceability links for reasons unrelated to content absence [9]. We show a specific instantiation of this failure in the security compliance context: content frequently exists but is not visible in the traceability surface — a phenomenon we formalize as *claim gaps*.

3. Dual-Source Knowledge Model

3.1 Normative Knowledge

Normative knowledge originates from external frameworks, standards, and regulations. In our model, it consists of external requirements — prescriptive statements that define what must be done, verified, or evidenced. Each requirement has:

- An identifier within its source framework (e.g., SSDF PW.5, ASVS injection_and_sanitization)
- A scope (what security concern it addresses)
- A granularity level (practice-level, requirement-level, or control-level)

Normative knowledge is authoritative but incomplete: it specifies *what* but rarely *how*, and it cannot capture practices that predate the framework’s publication.

3.2 Empirical Knowledge

Empirical knowledge originates from practitioner documentation: security manuals, coding guidelines, pipeline configurations, deployment procedures, training materials. In our model, the empirical corpus is decomposed into structural units — section-level segments, each annotated with:

- `document_role`: the type of document (requirements catalog, operational add-on, lifecycle user story, etc.)
- `normative_weight`: the epistemic authority (strong, medium, low)

- `heading_path`: the hierarchical position within the corpus

This annotation enables stratified analysis: coverage assessment can query the full corpus while distinguishing normative requirements (strong weight) from operational guidance (medium weight) from structural content (low weight/high authority).

3.3 The Integration Substrate

Both knowledge types are mapped to a shared ontological layer: AppSec Core v0 [1]. AppSec Core v0 provides 10 domain slices with 234 typed instances (ControlObjective, Practice, Mechanism, Artifact). The normalization works as follows:

- Normative requirements map to AppSec Core control objectives (what must be achieved)
- Empirical content maps to AppSec Core objectives, practices, mechanisms, and artifacts (the full type spectrum)

The richer mapping of empirical content is expected: practitioners document not only *what* they do (objectives) but *how* they do it (practices), *with what* (mechanisms), and *how they prove it* (artifacts). Frameworks typically prescribe only the first.

AppSec Core is not merely a mapping target — it is the semantic comparison space. Without a shared canonical layer, the normative mapping and the empirical mapping would not be commensurable: one would be expressed in SSDF/ASVS vocabulary and the other in the corpus’s internal terminology. AppSec Core makes them comparable by reducing both to the same set of typed objectives, enabling the coverage analysis in Stage 4.

4. Compilation Method

4.1 Overview

The compilation proceeds in five stages:

- | | |
|-----------------------------|--|
| Stage 1: Normative indexing | – register framework requirements |
| Stage 2: Empirical indexing | – decompose practitioner corpus into structural units |
| Stage 3: Dual mapping | – map both to AppSec Core independently |
| Stage 4: Coverage analysis | – compare normative surface against empirical projection |
| Stage 5: Editorial feedback | – feed results back into the corpus |

4.2 Stage 1: Normative Indexing

Each external framework undergoes a formal pilot — a systematic registration of its requirements into the compilation model. For each requirement, the pilot records: source framework, identifier, description, scope, and the AppSec Core slice(s) and objective(s) it maps to.

The pilot follows the mapping protocol defined in [1]: each requirement is mapped with a strength indicator (primary or secondary) and a human-authored rationale.

4.3 Stage 2: Empirical Indexing

The practitioner corpus is decomposed into structural units using automated indexing. In our case study, 4,139 units were extracted from a 15-chapter manual, each annotated with `document_role`, `normative_weight`, and `heading_path`. The distribution of document roles provides the empirical corpus structure:

Table 1. Distribution of document roles in the SbD-ToE empirical corpus (4,139 structural units).

Document role	Units	Share	Weight
addon	1,441	34.8%	medium
instantiated_policy	660	15.9%	medium
supporting_reference	483	11.7%	low
legacy_canon	321	7.8%	authority
aplicacao_lifecycle	316	7.6%	strong
intro	267	6.5%	low
kpi_catalog	186	4.5%	medium
maturity	156	3.8%	medium
advanced	130	3.1%	medium
policy_reference	109	2.6%	medium
requirements_catalog	68	1.6%	strong
Other	2	<0.1%	—
Total	4,139		

The empirical corpus is substantially larger and more diverse than the normative surface. Strong-weight units (requirements_catalog + aplicacao_lifecycle) constitute only 9.3% of the corpus; the remaining 90.7% is operational and contextual content — the “long tail” of empirical knowledge that frameworks do not capture.

4.4 Stage 3: Dual Mapping

Both normative requirements and empirical units are independently mapped to AppSec Core:

- Normative $\rightarrow$ Core: each framework requirement maps to one or more control objectives (the normative surface)
- Empirical $\rightarrow$ Core: each chapter of the practitioner corpus maps to one or more slices (primary + secondary); individual structural units are projected to objectives, practices, mechanisms, and artifacts (the empirical projection)

The two mappings are independent — neither influences the other. This independence is important: it prevents the empirical corpus from being shaped to fit frameworks, and vice versa.

4.5 Stage 4: Coverage Analysis

For each normative requirement r , the compilation assesses whether the empirical projection contains a structural unit u that satisfies three criteria:

Definition 1 (Coverage). A normative requirement r is covered by the empirical corpus C if there exists a unit $u \in C$ such that: (i) $u.content$ addresses the same security concern as r (scope overlap); (ii) u provides prescriptive or operational guidance, not merely a mention (actionability); (iii) $u.normative_weight$ is strong or medium (authority).

When coverage assessment reveals a gap, it is classified:

Definition 2 (Content Gap). A normative requirement r has a content gap if no unit $u \in C$ satisfies $covers(u, r)$. The concern is genuinely absent from the empirical corpus.

Definition 3 (Claim Gap). A normative requirement r has a claim gap if $\exists u \in C$ with $covers(u, r) = true$, but the corpus’s traceability surface T does not expose this coverage: $\neg \exists t \in T$ such that $maps(t, r)$. The content exists but is not visible.

Definition 4 (Cross-reference Gap). A normative requirement r has a cross-reference gap if a covering unit exists but in a different chapter than expected: $u.bundle_id \neq b_expected$.

The distinction between these gap types has practical consequences: content gaps require authoring; claim gaps require traceability repair; cross-reference gaps require navigation links. Conflating them leads to unnecessary authoring effort.

Critically, coverage assessment (Definition 1) is performed in the normalized objective space defined by AppSec Core [1], not directly between framework text and empirical text. The “same security concern” in criterion (i) is mediated by shared ControlObjectives: a framework requirement and an empirical unit both map to the same objective, and it is the objective-level match that establishes coverage. This ensures that coverage evaluation is independent of framework-specific vocabulary.

4.6 Stage 5: Editorial Feedback

The compilation’s gap classifications feed back into the empirical corpus:

- Claim gaps → traceability table corrections (add source reference)
- Cross-reference gaps → inter-chapter links
- Content gaps → authoring candidates

This feedback produces an improved version of the corpus ($C \rightarrow C'$) with better traceability. Subsequent compilation operates on the improved surface, reducing false gap reports. We term this the ontology-driven refinement loop.

4.7 Hard Rules

The compilation enforces three invariants:

1. `do_not_treat_canon_as_ground_truth` — the traceability tables in the corpus are editorial claims, not verified facts; verification is against the structural unit index
2. `do_not_write_directly_from_framework_into_manual` — framework requirements are normalized through AppSec Core, not copied into the corpus
3. `analysis_outputs_are_editorial_candidates_not_automatic_updates` — all changes require human review

4.8 Coverage Preservation Property

We define the property that gives the method its name:

Let r be a normative requirement, and let $normalize(r) = \{o_1, o_2, \dots, o_k\}$ be the set of AppSec Core ControlObjectives to which r maps (at the granularity and mapping semantics of the source framework).

The compilation is coverage-preserving if the coverage status assigned to r is determined by evaluating its normalized objective projection — that is, by assessing whether the empirical corpus satisfies the ControlObjectives in $normalize(r)$ — rather than by direct comparison between framework wording and empirical text.

In practice, coverage is assessed through mapped ControlObjectives according to the granularity of the source framework: SSDF maps at practice level, ASVS at thematic cluster level, SLSA at requirement level. The method does not require strict satisfaction of every mapped objective; it requires that the coverage *determination* operates in the normalized space.

Coverage preservation thus depends on the correctness of the normalization ontology (AppSec Core

[1]): if the mapping from r to $normalize(r)$ is wrong, coverage assessment is wrong regardless of the empirical content. This dependency is a strength (it makes evaluation principled and auditable) and a limitation (it makes the method only as good as the normalization layer).

5. Case Study

5.1 The Empirical Corpus

The case study uses the SbD-ToE (Security by Design — Theory of Everything) manual: a practitioner-authored security reference covering the full SDLC in 15 chapters. The manual was developed over 10+ years of hands-on AppSec engineering across multiple organizations. The manual was authored in Portuguese; field names, document identifiers, and heading paths throughout this paper reflect the corpus’s original naming.

Each chapter follows a canonical structure with four document types:

Table 2. SbD-ToE corpus chapter structure: four document types per chapter.

Type	Role	Weight
requirements_catalog	Typed requirement families (VAL, ERR, CFG, AUT, ACC, LOG, ...)	Strong
aplicacao-lifecycle.md	User stories operationalizing the chapter	Strong
addon/	Technical detail, operational guidance, examples	Medium
canon/	Normative structure, traceability tables	Authority

The corpus was decomposed into 4,139 structural units (Section 4.3).

5.2 The Normative Surface

Five external frameworks were compiled against the corpus:

Table 3. External frameworks compiled against the SbD-ToE corpus.

Framework	Items	Granularity
NIST SSDF v1.1	20	Practices
OWASP ASVS v5.0	36	Thematic clusters
SLSA Build Track v1.0	14	Requirements
CIS Controls v8.1 (AppSec scope)	8	Controls
CAPEC v3.9 (View 683)	13	Attack patterns
Total	91	

5.3 Methodology: Three Passes

The compilation was conducted in three passes of increasing rigor:

Pass 1 — Editorial baseline. The corpus’s traceability tables (25-rastreabilidade.md in each chapter) were read as proxies for content. This identified 36 apparent gaps across all frameworks.

Pass 2 — Direct reading. The actual addon files for the four most-pressured chapters (Security Require-

ments, Secure Development, IaC, Secure Deploy) were read directly. Many apparent gaps had substantive content in addons not cited in the traceability tables. This motivated the claim gap / content gap distinction.

Pass 3 — Systematic index verification. For each remaining gap, the structural unit index was queried programmatically: filter by chapter, document role, and content; record the matching source as a (document_role, normative_weight, heading_path) triple.

An LLM (Claude Sonnet, Anthropic, 2025) was used as an assisted query interface — formulating filter queries and surfacing candidate units from 4,139 entries. All coverage decisions were made by human inspection against Definition 1. The structural unit index was the source of truth. The LLM was not used for any text generation in this paper.

6. Results

6.1 Gap Classification

After three passes, the 36 initial apparent gaps were reclassified:

Table 4. Gap classification after compilation.

Gap type	Count	Share (approx.)	Remediation
Claim gap	22	61%	Traceability repair
Cross-reference gap	7	19%	Navigation links
Content gap	4	11%	Authoring needed
Scope exclusion	3	8%	None (deliberate)

Central finding: most apparent gaps were not content deficiencies but failures of the traceability surface to represent content that already existed in the corpus. The compilation method detected this because it operates on the full structural unit index (4,139 units), not on the traceability surface alone (approximately 15 traceability tables).

6.2 Framework Coverage

Table 5. Per-framework coverage after three-pass compilation.

Framework	Items	Covered	Claim gap	Content gap	Scope excl.
SSDF v1.1	20	16	2	1	1
ASVS v5.0 (clusters)	36	28	6	2	0
SLSA v1.0	14	11	2	0	1
CIS v8.1 (AppSec)	8	6	1	0	1
CAPEC v3.9	13	10	2	1	0
Total	91	71 (78%)	13 (14%)	4 (4%)	3 (3%)

After resolving claim gaps (traceability repairs), effective coverage rises to 84 of 88 in-scope items (95%) — within this case study, under the author-evaluated coverage protocol described in §5.3.

6.3 Content Gaps (Genuine)

Only four genuine content gaps were confirmed:

1. SLSA L3 hardened builds — the corpus covers L1/L2 but lacks hermetic/reproducible build specifics
2. CAPEC-691 (dependency metadata spoofing) — dependency confusion addressed; metadata spoofing not
3. CAPEC-443 (insider developer threat) — branch protection present; authorized-insider threat not distinct
4. SSDF PW.6 (compilation/interpreter security) — partially covered; compiler flags and interpreter hardening lack dedicated treatment

These represent genuine authoring needs not resolvable by traceability improvements.

6.4 Representative Claim Gap Reclassifications

SSDF PO.2 — Roles & Responsibilities. Appeared as a gap in three chapters. Content found in 00-fundamentos/roles-responsabilidades/ — 13 security roles with per-chapter responsibilities. Invisible because the chapter had no traceability table. Resolution: create traceability entry (no content authoring).

ASVS **secure_configuration_baseline**. Appeared as a gap in four chapters. Content found in: Cap. 02 addon/07 (CFG-001→007: debug off, env separation, vault, drift); Cap. 08 IaC principles addon (OPA policies, privilege minimum); Cap. 09 OPA/Kyverno admission addon. Three independently authored addons, never cross-referenced.

SSDF PW.7 — Code Review. Classified as “partial — not read in full, assumed partial.” Verified as covered: aplicacao_lifecycle (strong): US-02 Revisão de Código Segura — structured review with criteria and traceability.

6.5 Empirical Delta

The compilation reveals empirical content that extends beyond what the five frameworks require:

Category	Example	Framework status
Operational mechanisms	OPA/Kyverno admission policies, securityContext enforcement	Not prescribed by SSDF or ASVS
Evidence patterns	Requirement-linked test suites, validation coverage reports	Not required at this granularity
Role-based training curricula	Per-chapter training paths with knowledge validation	CIS-14 partially addresses; SSDF does not
Proportional application	Risk-level-dependent requirement activation (L1/L2/L3)	No framework specifies proportionality
Exception governance	Formal exception process with lifecycle, audit trail	Not prescribed at this operational level

This delta is the empirical knowledge that practitioners have accumulated but that normative frameworks have not (yet) formalized. It represents the value that empirical knowledge adds beyond framework compliance.

6.6 Refinement Loop Results

The compilation directly produced editorial corrections to the corpus:

Correction type	Count	Effect
Traceability table status reclassifications	22	Partial -> semantic
Source reference additions (Fonte verificada column)	29	Each cites a specific (role, weight, path) triple
Cross-chapter navigation links	7	Content now reachable from expected chapter
New traceability table created	1	Cap. 00 had no canon/ directory

The corpus improved not by writing new security content but by using the compilation to make existing content visible. This is the refinement loop in action.

7. Discussion

7.1 Why Empirical Knowledge Matters

The case study demonstrates that a mature practitioner corpus covers 95% of the normative surface (after claim gap resolution) while containing substantial content not captured by any framework. This suggests that:

- Frameworks lag practice: practitioners encounter and document security concerns before frameworks formalize them (e.g., CFG-001→007 documented in the internal corpus in 2023, based on corpus commit history; ASVS `secure_configuration_baseline` formalized in 2025)
- Frameworks abstract practice: they specify *what* but not *how*, leaving the operational layer to practitioners
- Empirical knowledge is not a luxury — it is the implementation substance that frameworks assume but do not provide

A compilation method that captures only normative knowledge loses this substance. A method that captures only empirical knowledge lacks coverage assurance. The dual-source model captures both.

7.2 The Claim Gap Phenomenon

The predominance of claim gaps (22 of 36 apparent gaps) has a structural cause. The corpus's traceability surface (canon/ directory) systematically references strong-weight units (`requirements_catalog`, `aplicacao_lifecycle`) but under-references medium-weight units (addons). Since addons constitute 34.8% of the corpus and contain the richest operational content, any analysis that reads only the traceability surface misses them.

This is not a flaw of this specific corpus — it is a structural consequence of epistemic stratification: when documentation is organized by authority level, the traceability layer tends to index the normative layer while the operational layer remains invisible. We conjecture that the claim gap phenomenon generalizes to other similarly stratified corpora, given the structural cause identified — but cross-organizational validation is needed to test this claim (see §8).

7.3 Coverage Adequacy, Not Completeness

The method assesses coverage adequacy — whether the empirical corpus addresses specific normative requirements — not domain completeness — whether the corpus captures all possible AppSec concerns. These are different properties:

- Coverage adequacy is measurable against a reference set (the frameworks)
- Domain completeness is not finitely verifiable

Our results (95% framework coverage, 4 content gaps) speak to the former. The latter would require validation against a broader set of practitioners and domains.

7.4 Generalizability

The compilation method is general: it requires (a) a normalization ontology, (b) a decomposed empirical corpus with role/weight annotations, and (c) an indexed normative surface. The specific ontology (AppSec Core), corpus (SbD-ToE), and frameworks (SSDF, ASVS, etc.) are instances. The method transfers to any domain with heterogeneous normative sources and practitioner documentation — though we have validated it only in the AppSec domain.

8. Limitations

Single empirical corpus. The case study uses one organization’s manual. Coverage rates and claim gap prevalence may differ for other organizations. We argue for structural transferability of the claim gap phenomenon but have not validated it cross-organizationally.

Author-evaluated coverage. The $\text{covers}(u, r)$ function was evaluated by the paper’s authors. Inter-rater reliability was not measured. We partially mitigate this by releasing a curated artifact set containing the first-wave pilot manifests, normalization outputs, and terminal comparison artifacts that support the reported case-study surface. The full internal structural unit index and all coverage decisions are not part of the public `v1.0.0` package.

Practice/cluster-level granularity. Coverage assessment operates at SSDF practice level and ASVS cluster level, not at individual requirement level. Finer-grained assessment would increase precision but also evaluation cost.

Temporal snapshot. Framework versions are captured as of early 2026. New versions require re-compilation.

Empirical delta not independently validated. The delta analysis (Section 6.5) identifies content extending beyond frameworks but does not verify whether this content is valuable or correct — only that it exists.

9. Conclusion

We have presented a method for compiling normative and empirical security knowledge into a unified, coverage-aware model. The method uses a normalized ontology as the integration substrate, enabling independent mapping of framework requirements and practitioner content to a shared semantic space.

The case study reveals three findings:

1. Claim gaps dominate apparent non-coverage. 22 of 36 apparent gaps were traceability failures

rather than content deficiencies. Compilation against the full indexed corpus — rather than the traceability surface alone — is necessary to distinguish the two.

2. Empirical knowledge substantially covers normative requirements. After claim gap resolution, 95% of in-scope framework requirements were covered, with only 4 genuine content gaps across 91 mapped items.
3. Empirical knowledge extends beyond frameworks. The practitioner corpus contains operational mechanisms, evidence patterns, proportional application models, and governance processes that no analyzed framework prescribes — the empirical delta.

The compilation method produces a measurable, auditable integration of both knowledge types. The ontology-driven refinement loop ensures that compilation outputs improve the source corpus, creating a virtuous cycle: the more the corpus is compiled against frameworks, the better its traceability becomes.

The method is coverage-preserving with respect to the normalized objective model: both knowledge sources — normative and empirical — are projected into the same canonical objective layer, and coverage is determined by evaluating that projection rather than by direct framework-to-text comparison. The preservation property (Section 4.8) depends on the correctness of the normalization ontology and the granularity of the mapping; it is only as reliable as the underlying normalization layer. This is a principled dependency: it makes the assumptions explicit and the evaluation auditable.

Knowledge compilation for security is not a one-time alignment exercise. It is an ongoing process in which normative frameworks provide coverage targets, empirical knowledge provides implementation substance, and a shared ontological layer enables the two to be systematically compared, integrated, and improved.

10. Artifact Availability

Curated supporting artifacts for this paper are available in the companion public repository at <https://github.com/sbd-ai-runtime/appsec-core-ontology-research>. For this paper, the relevant materials are organized under `papers/02-coverage-preserving-knowledge-compilation/artifacts/`, notably `papers/02-coverage-preserving-knowledge-compilation/artifacts/pilot_manifests/` and `papers/02-coverage-preserving-knowledge-compilation/artifacts/pilot_outputs/`, which contain the released first-wave pilot manifests and comparison outputs supporting the case-study surface. The same repository also contains this paper’s curated source under `papers/02-coverage-preserving-knowledge-compilation/source/` and its public PDF under `papers/02-coverage-preserving-knowledge-compilation/pdf/`.

References

- [1] P. Farinha, “AppSec Core: A normalized ontology for security requirements across heterogeneous frameworks,” preprint, 2026. [arXiv ID: TBD — submitted as companion preprint]
- [2] NIST. Secure Software Development Framework (SSDF) Version 1.1. SP 800-218. 2022.
- [3] OWASP. Application Security Verification Standard (ASVS) v5.0.0. 2025.
- [4] SLSA. Supply-chain Levels for Software Artifacts. Build Track v1.0. 2023.
- [5] NIST. Security and Privacy Controls for Information Systems and Organizations. SP 800-53 Rev. 5.

2020.

- [6] V. R. Basili, F. Shull, and F. Lanubile, “Building knowledge through families of experiments,” *IEEE Trans. Software Eng.*, vol. 25, no. 4, pp. 456–473, Jul./Aug. 1999, doi: 10.1109/32.799939.
- [7] A. Goknil, I. Kurtev, K. van den Berg, and J.-W. Veldhuis, “Semantics of trace relations in requirements models for consistency checking and inferencing,” *Software & Systems Modeling*, vol. 10, no. 1, pp. 31–54, 2011, doi: 10.1007/s10270-009-0142-3.
- [8] B. Ramesh and M. Jarke, “Toward reference models for requirements traceability,” *IEEE Trans. Software Eng.*, vol. 27, no. 1, pp. 58–93, Jan. 2001, doi: 10.1109/32.895989.
- [9] P. Mäder and J. Cleland-Huang, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, 2023, doi: 10.1007/s00766-023-00408-9.